

Producto de la Sesión 01

Conecta, inspecciona y explica un servicio Linux

Alumno: Edwin Darío Rivera

Entorno: Máquina virtual Ubuntu 26.04 LTS (VMware) — usuario edd

Servicio inspeccionado: ssh (OpenSSH)

Fecha: 20 de agosto de 2026

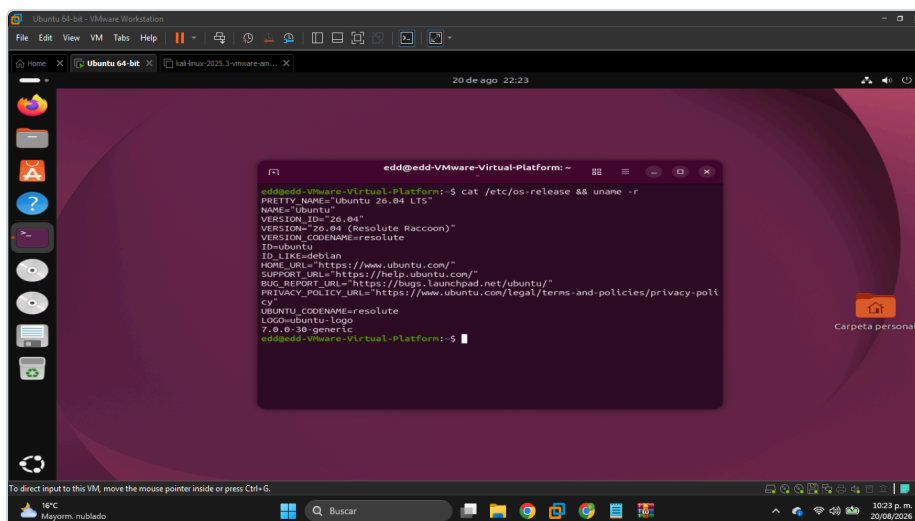
Bitácora breve de la práctica: para cada punto del checklist se muestra el comando ejecutado, la salida relevante, la interpretación de esa salida y la siguiente prueba a realizar.

1 Identificar distribución y kernel

COMPLETADO`/etc/os-release · uname -r`

Instrucciones: determinar qué distribución de Linux corre la VM y qué versión de kernel utiliza.

```
cat /etc/os-release && uname -r
```



Evidencia — salida de `/etc/os-release` y `uname -r` en la terminal de la VM.

Interpretación: el sistema es **Ubuntu 26.04 LTS ("Resolute Raccoon")**, distribución basada en Debian (`ID_LIKE=debian`). El kernel en ejecución es **7.0.0-30-generic**: la variante "generic" es la de propósito general (no low-latency ni cloud), la que trae por defecto el instalador estándar de Ubuntu Desktop/Server.

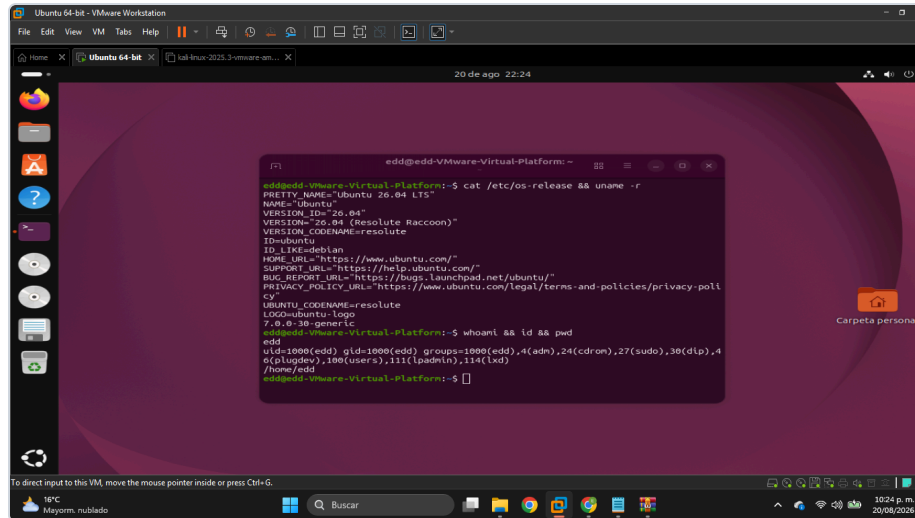
Siguiente prueba: comparar con `lsb_release -a` para confirmar que coincide con `/etc/os-release`, y revisar `apt list --upgradable` para saber si hay una versión de kernel más reciente disponible.

2 Confirmar usuario, grupos y ruta

COMPLETADO`whoami · id · pwd`

Instrucciones: confirmar con qué usuario se está trabajando, a qué grupos pertenece y en qué directorio se encuentra la sesión.

```
whoami && id && pwd
```



```
edd@edd-VMware-Virtual-Platform:~$ whoami && id && pwd
edd@edd-VMware-Virtual-Platform:~$ cat /etc/os-release && uname -r
PRETTY_NAME="Ubuntu 20.04 LTS"
NAME="Ubuntu"
VERSION_ID="20.04"
VERSION="20.04 (Focal Fossa)"
VERSION_CODENAME=focal
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=focal
edd@edd-VMware-Virtual-Platform:~$ whoami && id && pwd
uid=1000(edd) gid=1000(edd) groups=1000(edd),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),180(users),111(lpadmin),114(lxd)
/home/edd
edd@edd-VMware-Virtual-Platform:~$
```

Evidencia — usuario, grupos y directorio de trabajo actual.

Interpretación: la sesión corre como el usuario **edd** (uid=1000, gid=1000), un usuario estándar (no root) que pertenece a los grupos **adm**, **cdrom**, **sudo**, **dip**, **plugdev**, **users**, **lpadmin**, **lxd**. Pertenecer al grupo **sudo** es lo que le permite ejecutar comandos con privilegios elevados (como se hizo más adelante con **sudo ss**). El directorio actual es su home, **/home/edd**.

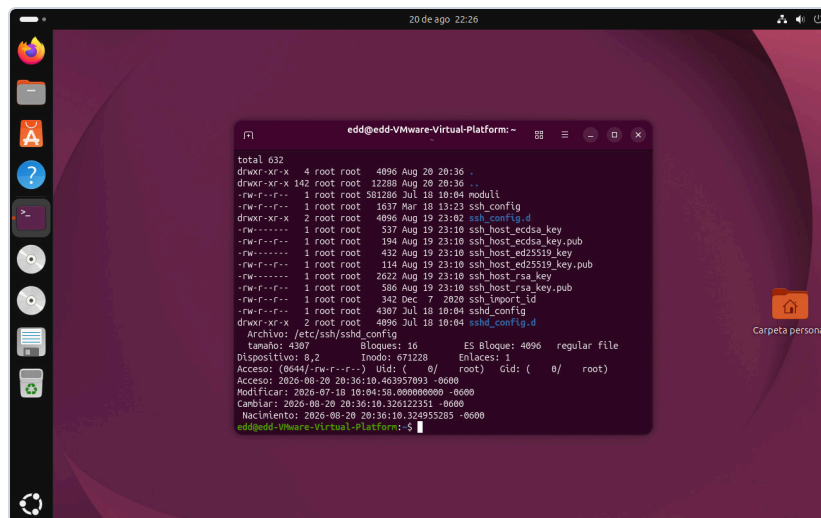
Siguiente prueba: ejecutar **groups edd** y **sudo -l** para ver exactamente qué comandos puede correr edd con privilegios elevados.

3 Examinar archivos y permisos COMPLETADO

ls -la · stat

Instrucciones: examinar el archivo de configuración del servicio (**sshd_config**) y revisar sus permisos y metadatos.

```
stat /etc/ssh/sshd_config
```



```
edd@edd-VMware-Virtual-Platform:~$ ls -la /etc/ssh/sshd_config
-rw-r--r-- 1 root root 4096 Aug 20 20:36 /etc/ssh/sshd_config
edd@edd-VMware-Virtual-Platform:~$ stat /etc/ssh/sshd_config
  Archivo: /etc/ssh/sshd_config
  Tamaño: 4096          Bloques: 16          ES Bloque: 4096    regular file
Dispositivo: 8,2        Inodo: 671228      Enlaces: 1
Acceso: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Acceso: 2020-08-20 20:36:10.463957003 -0600
Modificar: 2020-08-20 20:36:10.463957003 -0600
Cambiar: 2020-08-20 20:36:10.463957003 -0600
Nacimiento: 2020-08-20 20:36:10.463957003 -0600
edd@edd-VMware-Virtual-Platform:~$
```

Evidencia — metadatos y permisos de **/etc/ssh/sshd_config**.

Interpretación: el archivo pertenece a **root:root** con permisos **0644 (-rw-r--r-)**: root puede leer y escribir, mientras que el resto de usuarios (incluido edd) solo puede leerlo. Esto es correcto para un archivo de configuración de sistema — evita que un usuario normal pueda alterar la configuración del servicio SSH.

Siguiente prueba: correr `ls -la /etc/ssh/sshd_config` para confirmar el mismo permiso en formato corto, e intentar `nano /etc/ssh/sshd_config` sin `sudo` para comprobar en la práctica que `edd` no puede guardar cambios.

4 Observar proceso y servicio

PENDIENTE

ps · systemctl status

Instrucciones: localizar el proceso del servicio `ssh` y revisar su estado con `systemctl`.

```
ps aux | grep ssh
sudo systemctl status ssh
```

Falta evidencia de este punto. Aún no hay una captura de pantalla ejecutando estos comandos en la VM. Corre ambos comandos, toma la captura de la terminal y envíamela para completar esta sección con la interpretación (en el punto 5 ya se ve una pista: `ss -ltnp` muestra el puerto 22 en manos de `systemd`, no de un proceso `sshd` permanente — `systemctl status ssh.socket` probablemente lo explique).

Siguiente prueba: si `systemctl status ssh` no muestra el proceso escuchando, probar `systemctl status ssh.socket` y `systemctl status ssh.service` por separado, ya que en Ubuntu moderno el servicio puede estar activado por socket (activación bajo demanda).

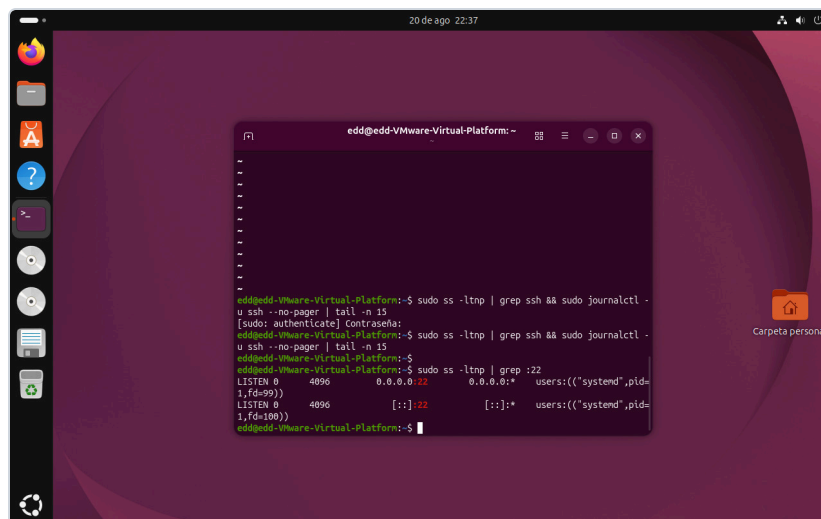
5 Relacionar puerto y evidencia

COMPLETADO

ss -ltnp · journalctl

Instrucciones: verificar en qué puerto escucha el servicio y relacionarlo con evidencia en los logs del sistema.

```
sudo ss -ltnp | grep ssh && sudo journalctl -u ssh --no-pager | tail -n 15 # intento 1: sin
salida
sudo ss -ltnp | grep :22 # intento 2: funcionó
```



Evidencia — sockets en escucha en el puerto 22 (IPv4 e IPv6).

Interpretación: filtrando por `:22` se confirma que hay dos sockets en **LISTEN**: uno en `0.0.0.0:22` (IPv4, todas las interfaces) y otro en `:::22` (IPv6). El propietario que reporta `ss` es `systemd (pid=1)`, no un proceso `sshd` corriendo permanentemente — esto indica que el servicio usa **activación por socket** (`ssh.socket`): `systemd` reserva el puerto y solo lanza `sshd` cuando llega una conexión real.

Nota sobre el intento 1: `ss -ltnp | grep ssh` no encontró la palabra "ssh" en la salida (el proceso aparece como "systemd", no "sshd"), así que `grep` terminó con estado de salida 1. Como el comando estaba encadenado con `&&`, `journalctl` nunca llegó a

ejecutarse — por eso la terminal no mostró nada más que el siguiente prompt. Filtrar directamente por :22 evitó el problema.

Siguiente prueba: repetir el log con `sudo journalctl -u ssh.socket --no-pager | tail -n 15` (nombre de unidad correcto) para relacionar el puerto abierto con eventos reales en el log.

Resumen de la bitácora

#	Punto del checklist	Comando principal	Estado
1	Identificar distribución y kernel	<code>cat /etc/os-release && uname -r</code>	Completado
2	Confirmar usuario, grupos y ruta	<code>whoami && id && pwd</code>	Completado
3	Examinar archivos y permisos	<code>stat /etc/ssh/sshd_config</code>	Completado
4	Observar proceso y servicio	<code>ps aux grep ssh · systemctl status ssh</code>	Pendiente
5	Relacionar puerto y evidencia	<code>sudo ss -ltnp grep :22</code>	Completado

Comandos y conceptos vistos

- Identificación de sistema: `cat /etc/os-release`, `uname -r`
- Identidad y permisos de usuario: `whoami`, `id`, `pwd`, grupo `sudo`
- Permisos de archivos: `stat`, notación octal (0644) y simbólica (-rw-r--r--), propietario root:root
- Sockets en escucha: `ss -ltnp`, activación de servicios por socket (`ssh.socket`)
- Logs del sistema: `journalctl -u <unidad>`
- Tuberías y códigos de salida: por qué `grep` sin coincidencias corta una cadena unida con `&&`

Criterio de entrega: bitácora breve con comandos, salida relevante, interpretación y siguiente prueba. El criterio no es cuántos comandos se ejecutaron, sino qué se pudo demostrar. Falta completar el punto 4 (proceso y servicio) antes de entregar esta práctica como definitiva.